ZhuLong: An Execution-Grounded LLM Agent for EDA Scripting

Yang Liu✉ Shiwei Hou Xiyuan Chen Yu Wang Sen Yuan Qirui Gan
Shao You Feifan Chen Wencheng Li Shuyang Hu Yongzhou Liu Emma Xia
Xiaojing Lu Hao Wang Fan Xu Yanfeng Li✉ Hongwen Li✉ Yiming Zhu
paul.liu@cxmt.com
yanfeng.li@cxmt.com
howie.li@cxmt.com
Changxin Memory Technologies, Inc.
China

Abstract

Design engineers lose the most time orchestrating flows and patching the gaps where closed EDA tools refuse to interoperate. We present **ZhuLong**, an execution-grounded LLM agent for PyAether, SKILL, and Tcl that closes a generate–execute–refine loop over unified MCP tools (API retrieval, documentation, sandbox execution) and is already deployed in production EDA flows, with a *grounding-gap framework* positing that EDA-scripting correctness is bounded by *observation fidelity*, not model scale. On **EDA-Scripting-Bench** (449 tasks, three languages, four commercial runtimes), ZhuLong reaches **76.9%** pooled Pass@1 versus **19.7%** (pure LLM)—a 57.2 pp observation-fidelity swing exceeding the 23.7 pp backbone range.

CCS Concepts

• **Hardware** → **Software tools for EDA**; • **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering** → *Software creation and management*.

Keywords

LLM Agents for EDA, EDA Scripting, Code Generation, Execution Grounding, Benchmarking

ACM Reference Format:
Yang Liu, Shiwei Hou, Xiyuan Chen, Yu Wang, Sen Yuan, Qirui Gan, Shao You, Feifan Chen, Wencheng Li, Shuyang Hu, Yongzhou Liu, Emma Xia, Xiaojing Lu, Hao Wang, Fan Xu, Yanfeng Li, Hongwen Li, and Yiming Zhu. 2026. ZhuLong: An Execution-Grounded LLM Agent for EDA Scripting. In *Proceedings of* . ACM, New York, NY, USA, 8 pages.

1 Introduction

Design engineers' daily work is less about writing fresh scripts than about orchestrating design flows and patching the gaps between EDA tools that refuse to interoperate—a flow that cannot run end-to-end is the single largest source of lost design time. Commercial EDA tools are closed: where they leave a gap, the only recourse is hand-written glue (a Makefile target, a Tcl driver, a SKILL wrapper), precisely the small intent-to-code synthesis at which LLMs excel. This reframes EDA scripting as the connective tissue that lets a general coding agent orchestrate flows natively and fill the breaks where tools stop; this paper asks which factor sets the ceiling of such an agent.

We present **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl: four unified MCP tools [13]—semantic and keyword API search, documentation lookup, and sandbox execution—close a generate–execute–refine loop over four commercial runtimes via a single `lang-host` pair, with offline **API self-exploration** as a bounded complement. We also give a **theoretical grounding** (Section 4): EDA correctness is situated (L4), and a Bayesian argument bounds the grounding gap by observation fidelity, not model scale—a refutable prediction, not a post-hoc explanation.

Contributions.

- **RQ1 (ceiling).** We formalize the grounding gap and measure it from both sides: the system-level axis (pure LLM, no observation → full system) spans 57.2 pp (LLM fixed), while the backbone axis spans 23.7 pp (harness fixed)—the system-level grounding range exceeds the backbone range (Section 4, 6).
- **RQ2 (necessary components).** Ablation isolates retrieval, execution, and belief update as jointly necessary, with toolchain-specific magnitudes that make them complementary rather than interchangeable; the cost columns add that full-fidelity read-back is also the cheaper feedback to operate; offline self-exploration is a bounded complement, not a substitute (Section 6).
- **RQ3 (toolchain generality). EDA-Scripting-Bench** (449 tasks, 4 runtimes, one schema) re-measures the same structure slice by slice; we specify the harness, schema, and assertion protocol in full (Section 5) so that the benchmark can be re-instantiated on other toolchains.

2 Background and Motivation

We target three widely-used EDA scripting interfaces across four commercial runtimes: PyAether for Empyrean Aether [7], SKILL for Cadence Virtuoso [16], and Tcl for Synopsys Custom WaveView and Cadence Innovus. Despite their prevalence, scripting for these environments poses challenges beyond what generic LLM code generators can address.

EDA scripting is fundamentally different. Prior LLM code generation succeeds where correctness is verified by unit tests [1, 3]. EDA scripting presents three distinct challenges: (i) APIs are

,

long-tailed, tool-specific, and poorly documented; (ii) correctness depends on side effects in a live, stateful database, not return values; and (iii) the community lacks a standardized benchmark. The first two are technical; the third is infrastructural.

Prior custom/analog scripting work stops short of execution. LayoutCopilot [11] targets analog *layout* (not scripting); ChaTCL [15] and RAG-EDA [14] generate Tcl scripts from static knowledge bases without executing them, so runtime errors go undetected. Execution feedback is not new—in the digital flow, ChatEDA [8], MCP4EDA [18], and AutoEDA [13] close the loop against synthesis and P&R state on open toolchains (Yosys, OpenLane, OpenRoad)—but none grounds against the live layout, schematic, or waveform database of custom/analog tools, the surface this paper targets. A recent survey [12] organizes EDA agents by handoff validity; ZhuLong targets the custom/analog scripting handoff, previously ungrounded, and our benchmark is the first for this setting (concurrent benchmarks focus on digital Tcl [19] or physical-design agents [10]).

Our approach. ZhuLong closes the loop from generation to execution to refinement, supplies a **theoretical grounding** (Section 4) explaining *why* execution is necessary, and provides the missing infrastructure: **EDA-Scripting-Bench** (Section 5).

3 System Design

3.1 Overall Architecture

ZhuLong extends Cline-CLI [4] with EDA-specific retrieval, documentation, and execution. The runtime is an **ICL-type autonomous agent**: it decides its own next action, keeps no explicit posterior, and maintains its belief state implicitly via in-context learning (Section 4). The policy stays commodity and unmodified, so gains are attributable to the interface, and the runtime's permission/hook mechanisms supply the enforcement points the fixed protocol relies on (Section 6.1). As in Figure 1, three components—the LLM agent runtime, the API knowledge base, and the EDA execution environment—are linked through four MCP tools, with offline **self-exploration** pre-augmenting the knowledge base.

3.2 Four MCP Tools

The four tools jointly realize the observation channel Ω and the effector of Section 4, each taking a `lang–host` pair that selects the runtime so one agent logic covers all four: `search_apis` retrieves APIs by semantic similarity (BGE-M3 embeddings [2] with FAISS [6], top-K, merged with BM25); `search_apis_by_keyword` retrieves by keyword (CamelCase plus BM25); `get_api_details` fetches documentation (parameters, returns, examples), augmented by self-exploration; and `run_code` executes code in an isolated headless sandbox and reads back a structured result (status, error log, stdout, return) for refinement, dispatching to resident or one-shot workers per runtime.

3.3 Agent Workflow

The agent iteratively refines its output through execution feedback [4]: reasoning over the instruction, retrieving and inspecting candidate APIs, generating and executing code, then re-planning based on the observed result. The agent is not constrained to a fixed sequence—it may skip retrieval when it already knows the API or conduct multiple rounds when initial results lack confidence—and terminates when it judges further attempts unlikely to succeed.

3.4 Offline API Self-Exploration (Complementary)

EDA API documentation is often incomplete, omitting parameter constraints, return structures, and error conditions. As Section 4 makes precise, retrieval enriches only the L1–L3 facts Ω can return, so self-exploration is complementary to—and bounded by—the L4 observation that requires acting on the live design; it is an efficiency mechanism, not the accuracy driver. Offline, the exploration agent reads documentation, identifies gaps, generates test scripts, executes them via `run_code`, and iteratively refines a behavioral model of constraints, return structures, error conditions, and side effects. Exploration stops when a per-API confidence score exceeds threshold τ, a round adds no new constraint, or a per-API budget M is exhausted; scripts are expert-gated to filter destructive operations, and the result is served at runtime via `get_api_details` with no overhead.

4 Theoretical Grounding: The Grounding Gap

This section develops the *grounding gap* framework, stated before the measurements of Section 6 so that its claims are predictions the experiments could refute rather than explanations fitted afterwards: EDA correctness is a *situated* property that cannot be guaranteed from offline knowledge, execution is the only channel that observes the current design state, and the resulting gap is bounded by observation fidelity, rather than model scale.

4.1 A Four-Level Hierarchy of Information

To separate what can be learned offline from what must be observed online, Table 1 distinguishes four levels: L1 *syntactic correctness*, L2 *physical plausibility*, and L3 *historical factuality* (documented, past-oriented, frozen at the training cutoff) are learnable offline, whereas L4 *situated truthfulness*—what is true *right now* in a specific environment—is not, defined by indexicality (truth depends on context and changes over time), non-guaranteeability from offline data, and real-time commitment. L4 must be *read* from the environment, not *recalled* from memory.

For EDA scripting, the situated state is the live design database—which cell/view is selected and what the layout or schematic contains now—plus the user's current intent (a user clarification is also an L4 observation). Our correctness criterion—code runs without error and all assertions over the mutated design pass—is L4: a well-formed (L1), plausible (L2) script invoking documented functions (L3) can still violate an undocumented binding contract, depend on internal selection state, or target the wrong cell. Such failures cannot be resolved offline; they must be *observed* by acting on the live design and reading back the result.

4.2 A Bayesian Formalization of the Grounding Gap

We cast grounding as decision-making under partial observability [9, 17]; the formulation is diagnostic rather than algorithmic—it

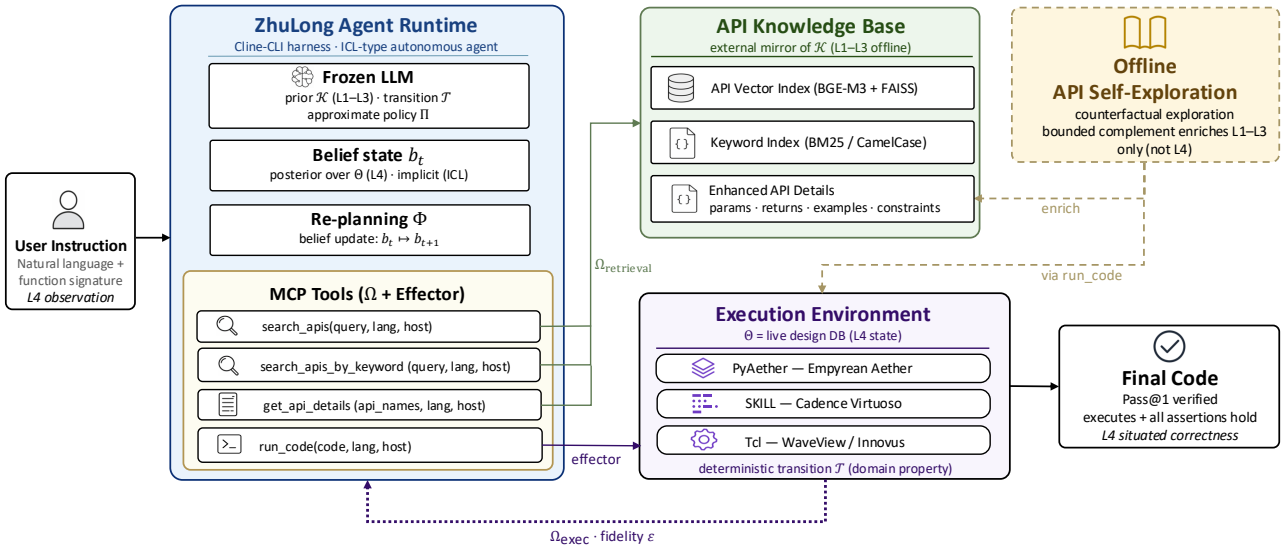


Figure 1: ZhuLong system architecture. The LLM agent runtime, the API knowledge base, and the EDA execution sandbox are linked through *four* unified MCP tools that dispatch on a single `lang`–`host` pair to *three* platforms (PyAether, SKILL, Tcl). Offline API self-exploration pre-augments the knowledge base (dashed); execution feedback is looped back for iterative refinement.

Table 1: The four levels of information

Level	Name	Core Question	Learnable Offline?
L1	Syntactic Correctness	Is this string well-formed?	Yes
L2	Physical Plausibility	Could this event happen?	Yes
L3	Historical Factuality	Did this event happen?	Yes
L4	Situated Truthfulness	Is this true *right now*?	**No**

makes precise what is missing (the observation likelihood Ω and the recursive update Φ) without prescribing tractable inference.

Let Θ denote the space of L4 states (the live EDA design database), o_t a runtime observation, a_t a generated code action, and $\mathcal{K}$ the offline knowledge (L1–L3) in the LLM's frozen parameters. The agent maintains a belief $b_t(\theta) \triangleq p(\theta_t \mid o_{1:t}, a_{1:t-1}, \mathcal{K})$ over the current L4 state, updated recursively by Bayes' rule:

$$b_t(\theta) \propto p(o_t \mid \theta_t) \int_\Theta p(\theta_t \mid \theta_{t-1}, a_{t-1})\, b_{t-1}(\theta_{t-1})\, d\theta_{t-1}, \quad (1)$$

where $p(o_t \mid \theta_t)$ is the *observation likelihood* Ω and $p(\theta_t \mid \theta_{t-1}, a_{t-1})$ is the *transition model* $\mathcal{T}$ supplied by the LLM's prior; actions are chosen by an approximate policy Π that minimizes expected cost under b_t, with C encoding the task assertions. Eq. (1) is the standard POMDP filter [9]; what is new here is not the filter but the *implementation* of Ω as retrieval plus execution read-back, which is this work's contribution to the POMDP stack.

The *grounding gap* is the divergence between the posterior achievable with runtime observations and the posterior from the offline prior alone:

$$D_{\text{grounding}}(t) \triangleq D_{\text{KL}}(p(\cdot \mid o_{1:t}, \mathcal{K}, \text{env}) \parallel p_{\mathcal{K}}(\cdot)), \quad (2)$$

where the right-hand side is the *pure prior* unconditioned on any observation. We now bound this gap.

Scenario: the transition is deterministic. In EDA, `run_code` executes a script whose effect on the design database is deterministic: the same action in the same state yields the same result, so $\mathcal{T}$ is exact—a property of the domain, not a simplifying assumption. The only mismatch is in the observation likelihood Ω: retrieval may return incomplete documentation, and execution read-back may truncate—precisely what ϵ captures. Model mismatch under estimated parameters is a recognized obstacle [5]; our setting is milder, with only Ω, not $\mathcal{T}$, possibly mismatched.

Claim (observation-fidelity bound). Let $\epsilon \triangleq \sup_\theta \|p(\cdot \mid \theta) - \hat{p}(\cdot \mid \theta)\|_{\text{TV}}$, the total-variation distance between the true and the agent-used observation likelihoods. We posit that $D_{\text{grounding}} \leq g(\epsilon)$, with $g(\epsilon) \to 0$ as $\epsilon \to 0$ and g *independent of the complexity of the prior* $\mathcal{K}$: the grounding gap is bounded by observation fidelity, rather than model scale. We state this as a *prediction the*

experiments can test rather than a theorem derived here; the next paragraph tests it from both sides.

Experimental correspondence (developed in Section 6). Here ϵ is exactly the fidelity of this paper's Ω implementation—retrieval quality plus execution read-back. The read-back ladder ($N' \to B \to F$, Table 3) varies ϵ with retrieval and the effector fixed; the backbone axis (Table 4) varies $\mathcal{K}$ with ϵ fixed. The experiments test whether the former axis dominates, as the claim predicts.

4.3 Design Principles

The formalization yields three architectural principles. **Decouple knowledge from belief:** adapt in the dynamic posterior b_t the Harness maintains, not by updating the frozen prior $\mathcal{K}$. **Optimize observation over the prior:** the gap is bounded by Ω's fidelity ($g(\epsilon)$, independent of model scale), so observation investment pays directly—full observation beats binary feedback on both accuracy and operating cost (Section 6.2)—while a richer prior gives diminishing returns. **Design for continuous belief updating:** Φ runs online ($b_t \mapsto b_{t+1}$); in the ICL-type agent (Section 3) what is bounded is not a round count but how much evidence belief can accumulate—a graded execution budget or a lagging read-back.

5 Benchmark Construction

5.1 Dataset: EDA-Scripting-Bench

EDA-Scripting-Bench is the instrument for both of this paper's comparisons: within one toolchain, the effect of each grounding component, and across toolchains, the effect of language structure. It covers PyAether, SKILL, and Tcl on four commercial runtimes, **449 tasks** in four slices (Table 2), all specified by one schema—a target language and runtime, a natural-language prompt, a function signature, and language-appropriate assertions—so a single harness (Section 3.3) scores all four platforms uniformly. Scoring is execution-based (a task passes only when the generated code acts correctly in the live runtime), and the cross-language slices are difficulty-controlled by construction (Section 5.3). Each task is a real break in a design flow where a closed tool stops and glue must be hand-written.

Table 2: Composition of EDA-Scripting-Bench

Slices and runtimes		
Slice	**Runtime**	**Tasks**
EDA-Eval-PyAether	Empyrean Aether	158
EDA-Eval-Skill	Cadence Virtuoso	157
EDA-Eval-WaveViewTcl	Synopsys Custom WaveView	77
EDA-Eval-InnovusTcl	Cadence Innovus	57
Total		**449**

5.2 The PyAether slice (EDA-Eval-PyAether)

The reference slice, **EDA-Eval-PyAether** (158 tasks), anchors every task to a concrete PyAether API; EDA experts curated each assertion to be self-contained and executable in a clean sandbox without vendor-internal context. Tasks span layout, schematic, and design-management workflows, compose several distinct APIs, and sometimes depend on a technology library, so passing reflects command of the tool's behavior, not surface retrieval.

5.3 SKILL and Tcl slices

The remaining slices are built so cross-language scores are comparable. **EDA-Eval-Skill** (157 SKILL tasks on Cadence Virtuoso) is *function-aligned* with PyAether: each task re-implements the same entry point and functional specification as its PyAether counterpart, holding difficulty fixed. The **Tcl** slices are graded by difficulty across two further toolchains: waveform measurement on WaveView (**EDA-Eval-WaveViewTcl**, 77 tasks) and floorplanning, placement, and routing on Innovus (**EDA-Eval-InnovusTcl**, 57 tasks), so the cross-language comparison (Section 6.2.4) attributes score differences to language structure rather than task hardness.

5.4 Evaluation Protocol

A task passes if the generated code executes in its target runtime sandbox without errors and all assertions hold; the primary metric is Pass@1 [3], reported as mean ± s.d. over five independent runs (reshuffled task orderings, fresh agent contexts), per slice and pooled over all 449 tasks.

6 Experiments

6.1 Experimental Setup

Research questions. The framework of Section 4 makes three testable claims, one per RQ: **RQ1 (ceiling)**—is accuracy bounded by observation fidelity Ω or the prior $\mathcal{K}$? (S1 varies Ω, S3 varies $\mathcal{K}$.) **RQ2 (necessary components)**—do observation, execution, and belief update each matter, and can one substitute for another? (S2 varies the effector and update budget.) **RQ3 (toolchain generality)**—does the structure reproduce on another toolchain? (S4 reruns S1–S2 on SKILL and Tcl.)

Benchmark and protocol. We evaluate on **EDA-Scripting-Bench** (449 tasks, Section 5) using **Pass@1**: one trace per task (the agent may iterate within it via Φ but is never restarted), a 2500-second timeout, and a `PreToolUse` hook that enforces a tool whitelist, blocks shell/code-search tools, and confines file access to a fresh directory—identical in every configuration. Each configuration runs five times (reshuffled order, fresh contexts), reported as mean ± s.d. Task cases derive from non-redistributable vendor documentation (unlikely in public pretraining), and no third-party agent can run these runtimes without the MCP bridge, so baselines are tool-registration ablations of the same harness—enabling single-variable attribution.

Configurations. We isolate components by tool registration (a disabled tool is unregistered and invisible; the agent policy is held fixed), giving five configurations (Table 3): *Pure LLM* (no retrieval, sandbox, or self-exploration), *RAG* (retrieval + self-exploration, no sandbox; = w/o-Sandbox), *w/o Self-Expl.* (retrieval + sandbox), *w/o Retrieval* (sandbox + self-exploration), and *ZhuLong (Full)* (all three on). S1 probes the observation axes at their extremes; S2 bounds Φ. All main experiments use DeepSeek-V4-Pro as the backbone; S3 evaluates five additional backbones under the identical harness.

Table 3: Main results on EDA-Scripting-Bench: Pass@1 (%) over five runs (mean ± s.d.), pooled over all 449 tasks and per slice. The five right-most columns report per-task operating cost over all 449 tasks (five runs, mean ± s.d.). The lower block varies the *read-back* Ω alone (retrieval and effector fixed); (F) coincides with the Full row.

	Pass@1 (%)				Per-task operating cost (449 tasks)				
Configuration	Overall (449)	PyAether (158)	SKILL (157)	Tcl (134)	Time (s)	run_code	Tool calls	In tok (k)	Out tok (k)
Pure LLM	19.7±1.4	14.3±2.1	33.2±2.3	10.1±1.1	183±9	—	3.6±0.1	72±2	6.3±0.7
RAG (retr.+self-expl., no sandbox)	68.6±1.6	71.1±4.6	52.2±1.9	84.6±1.9	226±9	—	16.9±0.7	272±8	8.6±0.3
ZhuLong w/o Retrieval	66.2±4.2	73.4±7.3	45.2±9.4	82.2±2.3	981±151	22.6±2.8	26.3±3.1	658±133	11.7±2.8
ZhuLong w/o Self-Expl.	74.5±1.8	77.0±4.1	**61.5±0.7**	86.7±3.0	645±132	12.6±1.4	29.5±2.9	722±116	11.5±2.3
ZhuLong (Full)	**76.9±2.3**	**82.5±2.0**	60.3±6.7	**89.9±2.0**	464±57	10.6±0.7	26.3±1.4	710±21	12.0±0.7
Read-back axis (retrieval + effector fixed, only what run_code *returns varies)*									
(N′) No read-back (prior only)	69.8±2.2	73.4±2.1	50.7±4.9	87.9±1.7	253±5	1.1±0.0	16.6±0.9	278±12	4.0±0.1
(B) Binary pass/fail	71.0±0.8	74.6±1.6	57.2±4.8	83.0† ±2.3	634±40	22.8±2.9	43.9±3.9	887±139	7.7±0.8
(F) Full observation (= Full row)	76.9±2.3	82.5±2.0	60.3±6.7	89.9±2.0	464±57	10.6±0.7	26.3±1.4	710±21	12.0±0.7

Bold marks the per-column best. — = no sandbox execution. † On Tcl, read-back inverts at the middle rung (short near-single-shot scripts solved largely from documentation; see Section 6.2.1). The Tcl column pools the two Tcl hosts, WaveView (77 tasks) and Innovus (57), by summing pass counts per run over 134 tasks. Time = net-active solve time (final-session active-seconds, excluding quota wait); run_code = execution calls per task; Tool calls = total tool invocations per task (search + details + execution + builtin); Input/Output tokens = input/output tokens per task (thousands), each averaged over all 449 tasks (five runs). Read-back-axis rows (N′, B) are costed on the same per-task, 449-task caliber.

6.2 Results and Ablations

Table 3 gives the five configurations pooled and per slice. Relative to the full system, removing execution costs 8.3 pp pooled, removing retrieval 10.7 pp, and removing self-exploration 2.4 pp; against the pure-LLM floor the full system gains 57.2 pp. Wilson 95% intervals on the pooled Pass@1 (n=2245): Full [75.11, 78.60], w/o-Self-Expl. [72.66, 76.26], w/o-Retrieval [64.22, 68.13], RAG [66.65, 70.49], N′ [67.87, 71.66], B [69.09, 72.84], Pure-LLM [18.11, 21.40]; the controlled $N' \to F$ read-back gap of 7.1 pp has 95% interval [4.52, 9.68] pp (SE= 1.32 pp). The same approximation gives the two RQ2 component gaps, which exclude 0—retrieval +10.7 pp [8.1, 13.3] and execution +8.3 pp [5.7, 10.9]—while self-exploration +2.4 pp [−0.1, 4.9], the binary-over-none read-back rung +1.2 pp [−1.5, 3.9], and the SKILL self-exploration difference −1.2 pp [−3.6, 6.0] straddle 0, so effects of this size are below this benchmark's power to certify, consistent with their complementary framing.[1] Where each component's contribution comes from is toolchain-specific (Section 6.2.4).

6.2.1 S1: Observation fidelity (RQ1). Principle 2 predicts that with the model fixed, accuracy is bounded by the fidelity of Ω. **Documentation axis.** Holding the agent loop, effector, and LLM fixed, removing retrieval costs 10.7 pp pooled, but highly toolchain-specific: on PyAether the read-back channel partially substitutes for retrieval (sandbox introspection via `dir()`/`inspect`/`__doc__` rises from 49.0% to 86.7% of traces once retrieval is removed, reconstructing the API knowledge retrieval would return). Tcl substitutes likewise through its own enumeration channels—WaveView's `info commands` returns the full command surface in one call, and Innovus's `get_db` probing enumerates attributes by trial and error—recovering the API surface but not its deep semantics, so removing retrieval there costs only 7.7 pp. SKILL's sandbox has no such enumeration channel (`apropos`/`describe` fail), leaving only `boundp`/`isCallable` existence checks on guessed names, so the API surface cannot be systematically rediscovered and retrieval there is not replaceable (15.1 pp). Retrieval is thus one *implementation* of Ω, not the channel itself.

[1] All difference intervals treat the two arms as independent and pool task-runs as i.i.d., both of which overstate variance; a paired, task-clustered analysis would only tighten them and strengthen the headline claims.

Read-back axis. The second axis is how much of the live state execution returns. We compare three closed-loop variants differing only here (Table 3, lower block): (N′) *no read-back*—`run_code` executes but returns nothing; (B) *binary feedback*—pass/fail only; (F) *full observation*—stdout, traces, per-assertion outcomes, and queried state (the default). The predicted monotone ordering N′ < B < F holds pooled and on the two largest slices, though on PyAether the middle rung is narrow (+1.2 pp, within one combined s.d.): a bare pass/fail verdict adds little actionable information, and the F rung carries almost the whole ladder (+7.9 pp of the 9.1 pp N′-to-F gap). On Tcl it inverts at the middle rung, because Tcl scripts are short and near-single-shot, largely solved from documentation, so a binary signal supplies no actionable information. Cost sharpens the reading (Table 3): binary feedback is *more* expensive than full observation in time, `run_code` calls, input tokens, and total tool calls (634 vs. 464 s, 22.8 vs. 10.6, 887k vs. 710k, and 43.9 vs. 26.3 per task), emitting fewer output tokens (7.7k vs. 12.0k)—without actionable error information the agent probes blindly—while no read-back collapses cost (253 s, 1.1 calls) together with accuracy. What the channel returns, not how often the agent acts, drives both. This is the ϵ-axis the claim of Section 4.2 predicts tracks accuracy; the N′-to-F gap operationalizes the grounding gap as an accuracy difference.

Self-exploration rung. The top rung adds constraints discovered offline (Section 3.4); per the framework this enriches only the L1–L3 facts Ω can return, so it should move the result less than the channel's own fidelity. Removing it (retrieval and effector kept) drops Pass@1 from 82.5% to 77.0% on PyAether (76.9% to 74.5% pooled), a bounded prior-side complement rather than a substitute. Self-exploration is cost-negative at runtime (cutting net-active solve time by 28.1%, see Table 3), whereas removing retrieval makes the agent fall back on sandbox introspection (2.1× the full system's time).

6.2.2 S2: The closed loop (RQ2). Principle 3 requires that actions reach the design through a causal interface and that Φ run continuously. **Effector.** Removing the sandbox drops pooled Pass@1 from 76.9% to 68.6% (−8.3 pp): without an effector, b_t receives no live-state evidence and the agent is a prior-only predictor. **Belief update.** We bound how often the agent *may* execute (the number of `run_code` calls per trace, denied at the tool boundary once

Table 4: Impact of LLM backbone (ZhuLong, full harness).

Backbone	PyAether (158)	SKILL (157)	Tcl (134)	Overall (449)	Δ
DeepSeek-V4-Pro	82.5±2.0	**60.3±6.7**	**89.9±2.0**	**76.9±2.3**	–
GLM-5.3-Flash	**86.8±1.6**	57.2±6.9	80.3±3.8	74.5±2.4	−2.4
GLM-5.2	83.3±3.1	54.4±4.5	82.7±1.6	73.0±2.1	−3.9
Kimi-K2.6	77.0±1.6	55.5±2.7	75.8±1.9	69.1±0.5	−7.8
DeepSeek-V4-Flash	60.1±5.5	47.0±1.8	70.4±3.1	58.6±2.3	−18.3
Doubao-Seed-2.0-Pro	63.8±1.4	42.5±3.4	53.3±3.4	53.2±1.9	−23.7

Pass@1 (%), five runs (mean ± s.d.); Tcl pools WaveView (77) and Innovus (57), by summing pass counts per run over 134 tasks; Δ vs. the primary backbone, pooled. Bold = per-column best. The strongest-to-weakest span is 76.9 − 53.2 = 23.7 pp.

spent) rather than forcing early submission, and vary the evidence along two axes: the budget k, and a *lagged* channel that returns the *previous* execution's state, leaving the agent one action behind the design. On PyAether, accuracy rises monotonically with the budget (63.8% at k=1, 69.1% at k=3, 77.7% at k=10, unsaturated); the lagged channel lands between k=3 and k=10 (73.0%)—stale evidence is worth markedly less than fresh—and tighter budgets leave traces unfinished (Converged drops from 99.1 to 76.3 at k=1).

6.2.3 S3: The prior axis (the prior-side contrast of RQ1). Principle 1 separates knowledge from belief: the frozen prior should bound what a harness can achieve without substituting for observation. Table 4 changes only $\mathcal{K}$ (the LLM) with the harness fixed; DeepSeek-V4-Pro is the primary backbone, yet the backbone remains decisive—even with retrieval, sandbox, and self-exploration in place, the strongest-to-weakest spread is 23.7 pp pooled. Read against Principle 2, this spread is what the $\mathcal{K}$ axis can move, and it does not substitute for observation: the system-level observation-fidelity axis moves the result by more than the entire backbone range (57.2 vs. 23.7 pp pooled), exactly the prediction of the claim in Section 4.2. GLM-5.3-Flash beats the primary on PyAether yet loses 2.4 pp pooled—slice-level snapshots can invert backbone rankings, itself an argument for the pooled protocol.

6.2.4 S4: The toolchain axis (RQ3). The four slices share one schema, assertion criterion, and harness, so S1–S2 re-runs slice by slice with nothing changed but the toolchain. Table 3 carries the comparison: the pure-LLM floor and the advantage of closed-loop configurations reproduce on every slice (the one exception being SKILL, where the full system trails its w/o-Self-Exploration variant by 1.2 pp), and the read-back ordering $N' < B < F$ holds on the two largest slices and pooled (narrowly at the middle rung on PyAether; the Tcl inversion is disclosed in Section 6.2.1). Component magnitudes are toolchain-specific—execution buys +8.1 pp on SKILL but only +5.3 on Tcl, and retrieval buys +15.1 pp on SKILL, +9.1 pp on PyAether, and +7.7 pp on Tcl, an ordering that tracks each host's introspective surface (Section 6.2.1)—but the structure (observation and execution jointly closing the loop) reproduces on every slice.

6.3 Error Analysis

ZhuLong passes 82.5% on PyAether (652/790 task-rounds), 60.3% on SKILL (473/785), 88.6% on WaveView (341/385), and 91.6% on Innovus (261/285; Table 3). We split each slice's misses into two stages: a *generation-stage* failure, where the round ends before any executable candidate is returned (the search budget saturates, or the final code is not recovered), and an *evaluation-stage* failure, where a candidate is returned but earns no pass from the grader. The generation/evaluation split is 29/109 on PyAether, 31/281 on SKILL, 1/43 on WaveView, and 1/23 on Innovus. We dissect the evaluation-stage failures below.

Across all four slices, the evaluation-stage failures that admit a clean attribution fall into four recurring patterns, all on solvable tasks whose corpus pass rate is nonzero across models or runs—not an impossible benchmark—where ZhuLong misreads an underspecified or invisible contract: **(i) Wrong internal API or contract**, a plausible but incorrect long-tail API, name, or field literal that survives until execution rejects it; **(ii) Unauthorized defensive guards**, a self-added validation or failure path that rejects input the specification never excluded; **(iii) Underspecified boundary and representation semantics**, an ambiguity the prompt leaves open that is resolved on the wrong side, flipping the verdict across runs; and **(iv) Host language and context contracts**, a type-template, headless-context, or canonical-form convention violated so the candidate dies before judging.

Concretely, these patterns surface as constraints documented nowhere and only weakly exposed by execution: PyAether-068 builds the cartoon-box highlight through the plausible-but-wrong `aeAddHighLightRect` where the oracle requires `GuCartBox(emyBoxF(...))`; WaveView-037 returns `N/A` on a single-rising-edge waveform and SKILL-037 rejects an empty hierarchy path—guards the specification never granted; Innovus-057 over-filters through an extra `.use=="signal"` predicate to report 1869 weight-2 nets against the oracle's 1872; and SKILL-008 hardcodes the schematic view type when opening a layout view, missed by every evaluated run although a direct probe proves the task passable.

These failures concentrate on contracts that are invisible in documentation and only weakly exposed by execution—the L4 invisibility of Section 4. On WaveView and Innovus, every model-side failure whose code generation succeeded is a flip between two interpretations of an underspecified point, and within a task all failing runs share one grader message: interpretation is a stable property of the task, so passing is a resampling event—precisely what the k-sample budget Φ buys.

7 Discussion

Section 4 elevates the error analysis of Section 6.3 from observation to structural principle: in domains with long-tailed, under-documented APIs, the grounding gap is bounded by observation fidelity, so execution channels—not larger priors—are binding, and fidelity is bought at a cost discount: full observation is both more accurate and cheaper to operate than binary feedback (Section 6.2.1), a paradigm that may generalize to other domain-specific scripting tasks.

Three limitations. Base-LLM performance varies widely (53.2%–76.9% pooled), and the benchmark size limits power to certify small effects, so the self-exploration gain (+2.4 pp) and the binary-over-none read-back rung (+1.2 pp) are complementary, not headline. Offline self-exploration costs a one-time pass per API, and the interactive bridge is single-threaded. Per-slice dispersions can be large (e.g., SKILL w/o Retrieval ±9.4), so slice-level comparisons need care; Pass@1 certifies only the effects its

assertions check, not full functional equivalence. The SKILL self-exploration regression (−1.2 pp) fits SKILL's smaller API surface—offline facts are more often redundant with retrieval and crowd out task-relevant evidence. Deployment metrics (adoption, task completion, engineer time saved) are subject to a separate disclosure review and will be reported in an extended version.

8 Conclusion and Future Work

ZhuLong reframes EDA scripting as the connective tissue that makes design flows runnable end-to-end: a general coding agent orchestrates flows natively and, where closed EDA tools leave gaps, synthesizes the missing glue. The execution-grounded loop and the grounding-gap bound on observation fidelity—not model scale—explain why this works; the full system reaches **76.9%** pooled Pass@1. Future work: (i) domain-specializing the harness, (ii) long-term memory and recursive self-improvement (RSI), (iii) multimodal grounding, and (iv) benchmarking design-flow orchestration.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732

[2] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216 [cs.CL] https://arxiv.org/abs/2402.03216

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374

[4] Cline Contributors. 2024. Cline: Autonomous AI Coding Assistant. https://github.com/cline/cline.

[5] Erick Delage and Shie Mannor. 2010. Percentile Optimization for Markov Decision Processes with Parameter Uncertainty. *Operations Research* 58, 1 (2010), 203–213.

[6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. arXiv:2401.08281 [cs.LG]

[7] Empyrean Technology. 2026. Empyrean PyAether: Full Custom Design Platform Ecosystem. https://empyrean.com.cn/solutions/index-128.html. Accessed: 2026-06-28.

[8] Zhuolun He, Haoyuan Wu, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. ChatEDA: A Large Language Model Powered Autonomous Agent for EDA. arXiv:2308.10204

[9] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial Intelligence* 101, 1-2 (1998), 99–134.

[10] Qiufeng Li, Rongqian Chen, Quan Cheng, Chengxuan Wang, Sizhe Tang, Wuxi Li, Duo Ding, Chia-Tung Ho, Haoxing Ren, David Z. Pan, Tian Lan, and Weidong Cao. 2026. PDAGENT-BENCH: Characterizing, Grounding, and Architecting LLM Agents for VLSI Physical Design. *arXiv preprint arXiv:2606.17253* (2026).

[11] Bingyang Liu, Haoyi Zhang, Xiaohan Gao, Zichen Kong, Xiyuan Tang, Yibo Lin, Runsheng Wang, and Ru Huang. 2025. LayoutCopilot: An LLM-Powered Multiagent Collaborative Framework for Interactive Analog Layout Design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 44, 8 (2025), 3126–3139. doi:10.1109/TCAD.2025.3529805

[12] Jiawei Liu, Peiyi Han, Yuntao Lu, Su Zheng, Fengyu Yan, and Bei Yu. 2026. Agentic Electronic Design Automation: A Handoff Perspective. arXiv:2606.19795

[13] Yiyi Lu, Hoi Ian Au, Junyao Zhang, Jingyu Pan, Yiting Wang, Ang Li, Jianyi Zhang, and Yiran Chen. 2025. AutoEDA: Enabling EDA Flow Automation through Microservice-Based LLM Agents. Uses MCP protocol for EDA automation. arXiv:2508.01012 https://arxiv.org/abs/2508.01012

[14] Yuan Pu, Zhuolun He, Tairu Qiu, Haoyuan Wu, and Bei Yu. 2025. Customized Retrieval Augmented Generation and Benchmarking for EDA Tool Documentation QA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 44 (2025). arXiv:2407.15353

[15] Yibo Rui, Yuanhang Li, Rui Wang, Ruiqi Chen, Yanxiang Zhu, Zhixiong Di, Xi Wang, and Ming Ling. 2025. ChaTCL: LLM-Based Multi-Agent RAG Framework for TCL Script Generation. In *Proceedings of the International Symposium of Electronics Design Automation (ISEDA)*. IEEE, New York, NY, USA, 736–742.

[16] Cadence Design Systems. 2024. *Virtuoso Studio SKILL Language Reference*. https://www.cadence.com

[17] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. 2005. *Probabilistic Robotics*. MIT Press, Cambridge, MA, USA.

[18] Yiting Wang, Wanghao Ye, Yexiao He, Yiran Chen, Gang Qu, and Ang Li. 2025. MCP4EDA: LLM-Powered Model Context Protocol RTL-to-GDSII Automation with Backend Aware Synthesis Optimization. arXiv:2507.19570

[19] Ning Xu, Zhaoyang Zhang, Senlin Shu, Qi Lei, Jiaqi Lv, Wensuo Wang, Tianhao Zhao, Chao Zhang, Zhaoliang Yang, Xiangyu Li, Zhaorui Su, Jingshan Li, and Xin Geng. 2026. iScript: A Domain-Adapted Large Language Model and Benchmark for Physical Design Tcl Script Generation. *arXiv preprint arXiv:2603.04476* (2026).